Министерство науки и высшего образования РФ Пензенский государственный университет
Кафедра «Системы автоматизированного проектирования»

ОТЧЁТ

по лабораторной работе №4
по дисциплине «Логика и основы алгоритмизации в ИЗ»
на тему «Бинарное дерево поиска»

Выполнили ст. гр. 25ВВВ4:
Кашкин И.А
Герасимова Е.А

Приняли:
Юрова О.В.
Деев М.В.

Пенза 2026

Задание 1. *Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.*

Задание 2. *Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.*

Задание 3. **Изменить функцию добавления элементов для исключения добавления одинаковых символов.*

1. Представленный код на Си был переписан на C++ с исправлением опечаток: в исходной функции добавления большие значения попадали в левое поддерево, что противоречит определению дерева поиска, поэтому теперь большие значения идут в правое поддерево, а остальные – в левое. Алгоритм поиска вводимого значения реализован с помощью рекурсии (функция search): на каждом шаге введённое число сравнивается с числом в узле дерева, и поиск продолжается в правом поддереве, если число больше, и в левом, если оно меньше. Если число совпало с числом в узле, функция возвращает указатель на этот узел, а если дошли до пустого узла (nullptr), значит, числа в дереве нет.

2. Функция подсчёта числа вхождений (countOccurrences) также реализована с помощью рекурсии. Для каждого узла счётчик count равен 1, если число в узле совпадает с искомым, и 0, если нет. К нему прибавляются результаты рекурсивных вызовов для левого и правого поддерева. Дерево обходится целиком, потому что при добавлении с повторениями одинаковые числа могут оказаться в разных ветвях.

3. В функцию добавления (createTree) добавлен параметр fNoDup, который включает режим без повторений. Если этот режим включён и добавляемое число равно числу в текущем узле, элемент не добавляется, а функция возвращает nullptr. При генерации случайного дерева такое число пропускается и генерируется следующее, поэтому в дереве всегда получается заданное количество элементов (не более 100).

Задание 4. **Оценить сложность процедуры поиска по значению в бинарном дереве.*

В худшем случае для поиска значения потребуется выполнить n = h операций, где h – глубина дерева. Например, если все числа вводятся по возрастанию или по убыванию, дерево превращается в цепочку (h = n), и мы ищем последнее число. Худший случай – O(h), для вырожденного дерева O(n).

В лучшем случае потребуется одна операция: искомый элемент стоит в корне дерева. Лучший случай – O(1).

В среднем в сбалансированном дереве на каждом шаге отсекается половина элементов. Для подсчёта сложности можно пойти с конца: на последнем шаге остаётся 1 элемент, на предыдущем – 2, до этого – 4, затем 8 и так далее до n элементов. Если в дереве n элементов и мы делаем h шагов, то $2^h = n$, откуда $h = \log_2 n$. Средняя сложность – O(log n).

Для сравнения: функция подсчёта вхождений обходит всё дерево, поэтому её сложность O(n).

Псевдокод:

```
СТРУКТУРА Node:
    data: целое число
    left: указатель на Node (по умолчанию null)
```

```
    right: указатель на Node (по умолчанию null)

// Добавление элемента в дерево поиска
ФУНКЦИЯ createTree(node, data, fNoDup):
    ЕСЛИ node == null:
        node = СОЗДАТЬ Node(data)
        ВЕРНУТЬ node
    ЕСЛИ fNoDup И data == node.data:
        ВЕРНУТЬ null // дубликат запрещён
    ЕСЛИ data > node.data:
        ВЕРНУТЬ createTree(node.right, data, fNoDup)
    ИНАЧЕ:
        ВЕРНУТЬ createTree(node.left, data, fNoDup)

// Вывод дерева на экран (обход «правое – узел – левое» с отступами)
ФУНКЦИЯ printTree(node, level = 0):
    ЕСЛИ node == null:
        ВЕРНУТЬ
    printTree(node.right, level + 1)
    НАПЕЧАТАТЬ (level * 2 пробела) + node.data
    printTree(node.left, level + 1)

// Генерация дерева случайных чисел
ФУНКЦИЯ genTree(node, size, fNoDup):
    ЕСЛИ size > 100:
        size = 100
    ПОКА size > 0:
        data = СЛУЧАЙНОЕ_ЧИСЛО(1, 100)
        ЕСЛИ createTree(node, data, fNoDup) == null:
            ПРОДОЛЖИТЬ // дубликат, генерируем заново
        size = size - 1

// Поиск значения в дереве
ФУНКЦИЯ search(node, data):
    ЕСЛИ node == null:
        ВЕРНУТЬ null
    ЕСЛИ data == node.data:
        ВЕРНУТЬ node
    ЕСЛИ data > node.data:
        ВЕРНУТЬ search(node.right, data)
    ИНАЧЕ:
        ВЕРНУТЬ search(node.left, data)

// Подсчёт числа вхождений значения
ФУНКЦИЯ countOccurrences(node, data):
    ЕСЛИ node == null:
        ВЕРНУТЬ 0
    count = 1, ЕСЛИ node.data == data, ИНАЧЕ 0
    ВЕРНУТЬ count + countOccurrences(node.left, data) + countOccurrences(node.right, data)

// Освобождение памяти
ФУНКЦИЯ clearTree(node):
    ЕСЛИ node == null:
        ВЕРНУТЬ
    clearTree(node.right)
    clearTree(node.left)
    УДАЛИТЬ node
```

```
    node = null

// Главный алгоритм
АЛГОРИТМ main:
    root = null
    ВВЕСТИ command // 0 – ручной ввод, 1 – генерация
    ВВЕСТИ fNoDup // 0 – дерево с повторениями, 1 – без повторений
    ЕСЛИ command == 0:
        ПОКА читается data:
            ЕСЛИ data == -1:
                ПРЕРВАТЬ
            createTree(root, data, fNoDup)
    ИНАЧЕ:
        ВВЕСТИ size
        genTree(root, size, fNoDup)
    НАПЕЧАТАТЬ «дерево:»
    printTree(root)
    ВВЕСТИ command // 0 – поиск значения, 1 – подсчёт вхождений
    ВВЕСТИ data
    ЕСЛИ command == 0:
        ЕСЛИ search(root, data) == null:
            НАПЕЧАТАТЬ «число не существует»
        ИНАЧЕ:
            НАПЕЧАТАТЬ «число существует»
    ИНАЧЕ:
        НАПЕЧАТАТЬ «вхождений: » + countOccurrences(root, data)
    clearTree(root)
```

Вывод:

В ходе лабораторной работы были реализованы алгоритм поиска введённого значения и функция подсчёта числа вхождений в бинарном дереве поиска, а также режим добавления элементов без повторений. В дереве поиска меньшие значения уходят в левое поддерево, а большие – в правое, поэтому на каждом шаге поиска одно из поддеревьев отбрасывается.

Сложность поиска зависит от формы дерева: в лучшем случае O(1), в среднем для сбалансированного дерева O(log n), в худшем случае, когда дерево вырождается в цепочку, O(n). Подсчёт вхождений обходит всё дерево и выполняется за O(n). Работа позволила освоить рекурсивные алгоритмы и динамическое выделение и освобождение памяти для узлов дерева.